Reporte Técnico de Actividades Práctico-Experimentales Nro. 001

1. Datos Generales

Asignatura	Sistemas distribuidos
Nombre del estudiante(s)	Alexander Sánchez Ariel Tandazo Jossibel Perez Leonardo Sánchez.
Ciclo	6to
Unidad	Envío de mensajes distribuido
Resultado de aprendizaje de la unidad	R2. Escribe un programa que realiza cualquier requerimiento de clasificación y conversión en unidades de mensajes, como paquetes, para comunicar información interesante entre dos hosts, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	Uno
Título de la Práctica	Análisis Colaborativo de las Arquitecturas Principales en un Sistema Distribuido
Nombre del Docente	José O. Guamán Q.
Fecha	29 de septiembre de 2026 02 de octubre de 2026
Horario	Martes 10h30 a 11h30 Viernes 07h30 a 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3

2. Objetivo(s) de la Práctica:

Analizar de manera colaborativa las arquitecturas fundamentales de los sistemas distribuidos (cliente-servidor, peer-to-peer, multicapa y microservicios), identificando sus características, ventajas, desventajas y casos de uso, mediante trabajo en equipo y el diseño de un caso de estudio

- Identificar las características estructurales y funcionales de las arquitecturas cliente-servidor, peer-to-peer, multicapa (n-tier), y microservicios en sistemas distribuidos.
- Comparar las ventajas, desventajas, escalabilidad y tolerancia a fallos de cada arquitectura mediante el llenado colaborativo de una matriz de análisis.
- Diseñar el diagrama de un caso de estudio propuesto aplicando la arquitectura distribuida más adecuada, justificando técnicamente la elección del equipo

3. Materiales y reactivos:

Diapositivas, y enunciados del problema:

- Hardware:
  - Computadora personal con conexión a internet.
- Software:
  - Procesador de texto.

4. Equipos y herramientas

Computadora personal con conexión a internet

Guía de práctica.

Plantilla de matriz comparativa

5. Procedimiento / Metodología

Fase 0 (10 min). Formación de equipos de 4 (asignación del docente).

Fase 1 – Investigación (50 min). Cada integrante investiga su arquitectura y completa su sección de la matriz (estructura, ventajas, desventajas, escalabilidad, ejemplos). El equipo elabora el diagrama de su arquitectura en draw.io.

Fase 2 – Socialización (30 min). Cada equipo expone 5 min. El resto completa la matriz comparativa con la información expuesta (una sola matriz, grupal).

Fase 3 – Caso de estudio (55 min). Cada equipo selecciona la arquitectura más adecuada para el enunciado, la justifica con mínimo 3 argumentos técnicos, elabora el diagrama de la solución e identifica dónde se aplican transparencia, escalabilidad y tolerancia a fallos.

Fase 4 – Reporte y entrega (25 min). Cada estudiante redacta sus conclusiones individuales; el equipo compila la matriz final y los diagramas. Entrega en la plataforma indicada.

Cliente - Servidor

Criterio	Cliente - Servidor
Estructura	En ésta arquitectura los clientes realizan solicitudes y un servidor central los procesa, proporcionando servicios o recursos. La comunicación se logra mediante red utilizando un modelo petición-respuesta.
Ventajas	- Facilita la administración y control centralizados. - Facilita la gestión de datos y recursos. - Permite implementar seguridad y permisos desde el servidor. - Facilita el mantenimiento y actualizaciones en el sistema
Desventajas	- Si existe un problema en el servidor todos los clientes se quedan sin servicio. - En caso de recibir demasiadas solicitudes, puede existir cuello de botella. - Necesita infraestructura y mantenimiento de servidor.
Escalabilidad	- Se puede escalar verticalmente, de forma que se aumentan los recursos del servidor mediante el hardware, y se puede escalar horizontalmente agregando varios servidores, balanceo de carga o réplicas
Ejemplos reales	**Aplicaciones web:** Un cliente mediante un navegador solicita datos y páginas a un servidor web **Correo electrónico:** Un cliente de una empresa de correos electrónicos se comunica con los servidores de la compañia para comprobar si le llegaron oficios.

Peer-to-Peer

Criterio	Peer-to-Peer
Estructura	No hay servidor central. Cada equipo (nodo) actúa como cliente y servidor a la vez, compartiendo recursos directamente con los demás.
Ventajas	- No depende de un solo equipo, así que es más resistente a fallos. Los costos son menores al no requerir un servidor potente. Cada nuevo usuario aporta recursos. - Mayor disponibilidad de los datos: como el contenido está copiado en varios nodos, se puede seguir descargando aunque algunos equipos se desconecten.
Desventajas	- Es más difícil de administrar y asegurar, porque no hay control central. - Seguridad débil: cada equipo se protege por su cuenta, así que un nodo infectado puede propagar virus o archivos maliciosos a los demás. - Respaldo y control difíciles: como los datos están repartidos, no hay un lugar único para hacer copias de seguridad ni para controlar quién accede a qué. Si un nodo se apaga, sus archivos pueden dejar de estar disponibles.
Escalabilidad	Alta: al sumar usuarios también se suman recursos (ancho de banda, almacenamiento).
Ejemplos reales	BitTorrent, blockchain (Bitcoin, Ethereum), IPFS, y en sus inicios Skype.

Multicapa

Criterio	Multicapa
Estructura	Su estructura se basa en una organización jerárquica en capas independientes, cada una con una responsabilidad específica y comunicación definidas entre ellas.
Ventajas	En este caso la arquitectura Multicapa nos facilita el mantenimiento y las pruebas, al permitirnos modificar una capa sin afectar directamente a las demás, favoreciendo la reutilización de componentes.
Desventajas	La desventaja que puede haber es que el sistema puede aumentar su dificultad debido a la cantidad de capas y si llega a haber una mala implementación puede hacer que las capas dependan demasiado unas de otras. La comunicación entre capas puede generar sobrecargas y afectar el rendimiento.
Escalabilidad	Permite incrementar la capacidad del sistema mediante la ampliación de los recursos de las diferentes capas. Las capas pueden escalar de manera independiente según la demanda, facilitando la distribución de carga y el crecimiento progresivo del sistema.
Ejemplos reales	Sistemas bancarios, Sistemas de gestión empresarial, Sistemas para usos académicos.

Microservicios

Criterio	Microservicios
Estructura	Se crea como un grupo de servicios pequeños, independientes y débiles que se acoplan. Cada servicio implementa una sola capacidad de negocio, como pedidos o pagos, que se ejecutan en su propio proceso ya que administra su propia base de datos y son independientes. Cada servicio usa la red para comunicarse con las interfaces bien definidas o de manera simultánea a través de (APIs REST o gRPC) o asíncrona que se da por medio de mensajes basados a eventos Se compone por Clientes, API Gateway, Microservicios, Base de datos por servicio, Servicio discovery, Menssage broker, Orquestador, Monitoreo y registro centralizado

Ventajas	Tiene un despliegue independiente, ya que el servicio se actualiza sin detenerse ni volver a desplegar toda la aplicación, solo se replican los servicios que reciben más carga, la caída de un servicio no detiene a los demás, cada servicio puede usar el lenguaje y la base de datos más adecuados, los equipos pequeños son dueños de servicios concretos y avanzan en paralelo.
Desventajas	Son más componentes que desplegar, las llamadas entre servicios viajan por la red y pueden fallar o demorar. La Consistencia de datos, al tener bases de datos separadas, las transacciones distribuidas requieren patrones como Saga y consistencia eventual. Su depuración y pruebas, sigue una petición a través de varios servicios y exige trazabilidad distribuida, Requisitos de madurez se necesita automatización (CI/CD, contenedores, orquestación) para que sea viable.
Escalabilidad	Su escalabilidad es de manera horizontal en vez de aumentar su capacidad de un único servidor, se levantan más de una instancias del servicio que lo necesita y un balanceador divide las peticiones entre ellas. Los orquestadores como Kubernetes automatizan este proceso con el autoescalado según el uso de CPU o el número de solicitudes. Como cada servicio no guarda estado de sesión (stateless), las nuevas instancias pueden atender tráfico inmediatamente.
Ejemplos reales	Tanque Anti-Derrame

Diagramas en Draw.io
Cliente - Servidor

Peer-to-Peer

Multicapas

Microservicios

Fase 5: Socialización y matriz comparativa

1. Cada equipo expone su arquitectura durante 5 minutos (proyector)
2. Durante las exposiciones, todos los equipos llenan la matriz comparativa con la información de las cuatro arquitecturas:

GRUPO	Criterio	Cliente-Servidor	Peer-to-Peer	Multicapa	Microservicios
	Estructura	La estructura está conformada por nodos tanto como el nodo servidor y el nodo cliente.	La estructura se basa en una red de computadoras		La aplicación se divide en servicios pequeños que trabajan juntos. Cada microservicio se encarga de una función concreta del negocio y puede ejecutarse de manera independiente.
GRUPO 1 ...	Ventajas	Esta arquitectura nos proporciona las siguientes ventajas: Organización de las funciones	Al ser un arquitectura descentralizada las ventajas que posee son varias tales	Esta arquitectura ofrece varias ventajas debido a su organización jerárquica. Esto	Se puede modificar, corregir y desplegar un servicio sin tener que desplegar nuevamente toda la aplicación. Como cada servicio se

		entre cliente y servidor. Centralización de la información y los recursos.	como: una mejora en la escalabilidad al permitir a las redes peer-to-peer albergar un número ilimitados de nodos sin afectar el tiempo de respuesta de la red.	permite la separación de responsabilidades, permite el desarrollo en paralelo y facilita la mantenibilidad y modularidad de los sistemas que la implementan.	concentra en una función, resulta más fácil entender su código y darle mantenimiento.
	Desventajas	También dentro de esta arquitectura nos encontramos las siguientes desventajas: Problemas de escalabilidad cuando aumenta considerablemente el número de clientes.	Una alta volatilidad en los nodos que al conectarse y desconectarse de forma continua incrementa la posibilidad de fallos en el sistema y provoca la pérdida momentánea de disponibilidad de los recursos	Entre ellos se destacan la alta complejidad de gestión y orquestación de los múltiples servicios e interfaces, además que a veces simples cambios en los datos puede requerir actualizar el esquema de la base de datos.	Tener varios servicios separados también hace más complicado el funcionamiento de la aplicación. Si una operación necesita comunicarse con varios servicios, esas llamadas por red pueden causar demoras. Cuando los datos están repartidos entre servicios, se necesitan mecanismos adicionales para mantenerlos consistentes.
	Escalabilidad	Esta arquitectura es escalable mediante el aumento de los recursos del servidor. También es posible incorporar varios servidores y distribuir las solicitudes entre ellos para atender un mayor número de clientes.	Alta capacidad de escalado global permitiendo albergar potencialmente un número ilimitado de nodos a nivel mundial.	Tiene factores que favorecen la escalabilidad, como la distribución de carga de trabajo entre múltiples nodos o servidores y la eliminación de puntos únicos de fallo que permiten añadir más nodos sin degradar drásticamente el rendimiento global.	Tener varios servicios separados también hace más complicado el funcionamiento de la aplicación. Si una operación necesita comunicarse con varios servicios, esas llamadas por red pueden causar demoras. Cuando los datos están repartidos entre servicios, se necesitan mecanismos adicionales para mantenerlos consistentes.
	Ejemplos reales	Sistema Universitario, ya que el estudiante utiliza un navegador como cliente y el servidor de la universidad procesa las consulta, además esta arquitectura se	BitTorrent, Kazaa, Gnutella plataformas académicas como DisCoP, CompuP2P.	Las aplicaciones bancarias. debido a Esto que manejan una capa de artículos empresas adoptaron microservicios como que presentación que es la	Netflix y SoundCloud aparecen en los artículos como empresas que adoptaron microservicios. SoundCloud empezó con una aplicación monolítica y tuvo dificultades de escalabilidad y mantenimiento.

		puede presentar tambien en tiendas en línea como Amazon.		aplicación propiamente dicha con la que el usuario interactúa, pero también se accede a microservicios para validar la lógica del negocio	
GRUPO 2 ...	Estructura	Aplicación dividida en dos componentes principales: clientes y servidores, el cliente realiza la solicitud de servicios o datos, mientras que el servidor proporciona los recursos	Los nodos se conectan entre sí: estructurada, no estructurada, híbrida. descentralizada/pura.	Separación por responsabilidad	Divide y vencerás, servicios aislados que se comunican entre sí. Servicio, gateway u orquestadores
	Ventajas	Centralización de datos en el servidor, lo cual garantiza que todos los clientes accedan a la misma información Reducción de conflictos de información	No depende de un solo servidor (tolerante a fallos) Comparten recursos entre sí. Optimización	Permite atomización Permite que los errores no afecten al todo Testability	Tolerancia a fallos El servicio puede funcionar aunque una parte se esté actualizando
	Desventajas	Dependencia de servidor, esto implica que los clientes se vean afectados si el servidor presenta algún fallo o caída que interrumpa el flujo de trabajo de una organización, empresa, generando pérdidas económicas.	Difícil de localizar fallas en nodos. No todos los nodos son iguales ni tienen la misma sincronización	El antipatróñ architecture sinkhole Riesgo de bloqueo tecnológico	La desventaja principal es el mantenimiento Consistencia de datos Coste de mantenimiento
	Escalabilidad		Entre más nodos mejor, menor saturación y más tolerancia.	Nivel intermedio entre monolito y microservicios	Se levanta más de una instancias del servicio que lo necesita y un balanceador divide las peticiones entre ellas
	Ejemplos reales	Netflix, Youtube	Criptomonedas, BitTorrent.	ConStore y libreriás de Java, Health Watcher.	Spotify J.P Morgan

GRUPO 3 ...	Estructura	En ésta arquitectura los clientes realizan solicitudes y un servidor central los procesa	Todos son iguales cliente-servidor	Capas de presentacion, acceso de datos, datos	Api Gateway que se conecta a muchos microservicios
	Ventajas	Centraliza la información varios clientes usen varios servicios Todo en un servidor	Descentralizado, tiene una comunicación directa, tiene una alta tolerancia a faltas de distribución	Manejo individual	Cada microservicio puede ser actualizado independientemente Más fácil de manejar a nivel de daos
	Desventajas	SI algo falla todo cae Si no está bien equipado se satura	fireware bloquea estas redes	En sistemas pequeños es innecesario Muchas capas da complejidad a sistemas sensillos	Se puede volver un bucle entre servicios Multiples base de datos Costos
	Escalabilidad	Vertical: aumentar hardware Horizontal: Nodos	entre mas nodos mas eficiencia infinitamente	Banco	Horizontel: muchos microservicios Vertical: cada microservicio crecer sin afectar al resto
	Ejemplos reales	Bancos	Gnutella		Amazon
GRUPO 4 ...	Estructura	Servidor que responde sus solicitudes	Todos son iguales cliente-servidor	Organización en 3 niveles: capa de presentación, Capa media de lógica de negocio y capa de almacenamiento	Enrutamiento segmentado, Frontend →API gateway-Microservicios independientes
	Ventajas	El desacoplamiento, donde se conecta múltiples clientes simultáneamente	Descentralizado, tiene una comunicación directa, tiene una alta tolerancia a faltas de distribución	Estricta en división de tareas.	Independencia total Libertad para lenguaje de programación
	Desventajas	Congestion si no se posee los recursos necesarios	Alta complejidad de software, y tiene riesgo de saturación y alto trafico	En sistemas pequeños puede provocar una latencia demasiado elevada.	Mayor riesgo de filtración de datos Alto consumo sumado de recursos
	Escalabilidad	Incrementar mas recursos.	Puede tener una alta escalabilidad, por que puede llegar a ser infinita, si se unen mas usuarios.	Es fácil añadir nuevos nodos por capa ya que no se afecta a las demás, pero si se tiene una estructura muy compleja de capas habrá tardanza en los	Se limita por el presupuesto ya que se puede añadir más nodos.

				resultados.	
	Ejemplos reales	Sistema universitario U de Cuenca, ejemplo real es solventar dudas con un docente	Redes de Criptomonedas	Sistemas bancarios o sistemas academicos	Las billeteras digitales, la pasarella de pagos, su autenticación
GRUPO 5 ...	Estructura	En ésta arquitectura los clientes realizan solicitudes y un servidor central los procesa	Descentralizado, tiene una comunicación directa, tiene una alta tolerancia a faltas de distribución	Separación por responsabilidad	Divide y vencerás, servicios aislados que se comunican entres sí. Servicio, gateway u orquestadores
	Ventajas	Centraliza la información varios clientes usen varios servicios Todo en un servidor	Descentralizado, tiene una comunicación directa, tiene una alta tolerancia a faltas de distribución	Permite atomización Permite que los errores no afecten al todo Testability	Tolerancia a fallos El servicio puede funcionar aunque una parte se esté actualizando
	Desventajas	SI algo falla todo cae Si no está bien equipado se satura	Alta complejidad de software, y tiene riesgo de saturación y alto trafico	El antipatróñ architecture sinkhole Riesgo de bloqueo tecnológico	La desventaja principal es el mantenimiento Consistencia de datos Coste de mantenimiento
	Escalabilidad	Vertical: aumentar hardware Horizontal: Nodos	Puede tener una alta escalabilidad, por que puede llegar a ser infinita, si se unen mas usuarios.	Nivel intermedio entre monolito y microservicios	Se levanta más de una instancias del servicio que lo necesita y un balanceador divide las peticiones entre ellas
	Ejemplos reales	Bancos	Criptomonedas, BitTorrent.	ConStore y libreriás de Java, Health Watcher.	Spotify J.P Morgan
GRUPO 7 ...	Estructura	Un servidor central atiende múltiples clientes.	Todos los nodos son iguales y comparten recursos entre sí.	Separación por capas: presentación, lógica y datos.	Conjunto de servicios pequeños e independientes que se comunican por APIs.
	Ventajas	Fácil de administrar y controlar datos.	Alta tolerancia a fallos, sin servidor central.	Mantenimiento sencillo y modularidad.	Alta flexibilidad y despliegue rápido.
	Desventajas	Si el servidor falla, todo el sistema se detiene.	Difícil de controlar y asegurar.	Complejidad en la comunicación entre capas.	Requiere buena gestión de comunicación y monitoreo.
	Escalabilidad	Escalable verticalmente (mejor hardware).	Escalable horizontalmente (más nodos).	Muy escalable por capas independientes.	Altamente escalable y adaptable.

	Ejemplos reales	Correo electrónico, bases de datos, páginas web.	BitTorrent, Skype, blockchain.	Aplicaciones web empresariales, sistemas bancarios.	Netflix, Amazon, Spotify.

6. Resultados esperados:

Al finalizar la práctica, el estudiante deberá haber obtenido y documentado lo siguiente:

- Documento de investigación de la arquitectura asignada (definición, componentes, ventajas/desventajas, escalabilidad y 2 ejemplos reales).
- Un diagrama digital de la arquitectura asignada (draw.io o similar).
- Matriz comparativa grupal completa (integrando las cuatro arquitecturas con los aportes de todos los equipos).
- Diseño del caso de estudio: diagrama + justificación técnica (mínimo 3 argumentos) + identificación de transparencia, escalabilidad y tolerancia a fallos.
- Reporte individual breve (máx. 1 página) con conclusiones sobre qué arquitectura considera más versátil y por qué.

7. Preguntas de Control:

¿Qué es un sistema distribuido y cuáles son sus características fundamentales que lo diferencian de un sistema centralizado?

Un sistema distribuido es la colección de computadoras independientes separadas físicamente pero conectadas a través de una red, y trabajan juntas como si fueran un solo sistema. Tienen su propio procesador y memoria, pero se coordinan para realizar tareas.

Se diferencian de un sistema centralizado:

- Varias máquinas en vez de una sola: en el distribuido hay muchas computadoras colaborando; en el centralizando solo hay una.
- Si falla, el resto puede seguir: en el distribuido no se cae todo el sistema si se apaga una máquina. En el centralizado, si se daña la única computadora, se para todo.
- Los recursos repartidos: cada máquina tiene sus propios archivos o programas, pero se comparten. En el centralizado todo está guardado en un solo lugar.

Explique la diferencia de comunicación entre la arquitectura cliente-servidor y la arquitectura peer-to-peer.

Arquitectura cliente-servidor:

Hay dos roles claros: los clientes (los que piden cosas, como tu celular o computadora) y el servidor (el que responde y guarda información.

La comunicación es “de uno a muchos”: todos los clientes hablan con el servidor. El servidor es el centro y controla todo.

Ejemplo: cuando entras a una página web, tu navegador es el cliente y el servidor de la página te manda la información.

Arquitectura peer-to-peer:

Todas las computadoras son iguales(se llaman “pares” o “peers”). No hay un servidor principal.

Cada una puede pedir y también dar información a los demás.

La comunicación es directa entre ellas, sin pasar por un centro.

Ejemplo: programa para compartir archivos donde cada persona que tiene el archivo lo comparte con otros.

¿Qué rol cumple el middleware en un sistema distribuido y en qué capa se ubica?

El middleware es como un “traductor” o “puente” que facilita que las diferentes computadoras de un sistema distribuido se entiendan y trabajen juntas.

El rol principal es ocultar la complejidad de la red: hace que los programas no tengan que preocuparse por cómo se conectan las máquinas, como se envían mensajes o como se encuentran unos a otros.

Se ubica en una capa intermedia: entre el sistema operativo (la base de cada computadora) y las aplicaciones que usa el usuario. En el medio, para conectar todo de forma más fácil.

En la arquitectura de tres capas (3-tier), ¿qué función cumple cada capa y por qué separarlas mejora el mantenimiento?

En la arquitectura de tres capas, la capa de presentación o interfaz de usuario captura las interacciones del usuario y muestra los datos. La capa de negocio procesa la información, ejecuta las reglas del negocio y valida las operaciones. La capa de datos gestiona el almacenamiento y las consultas de la base de datos.

Separarlas mejora el mantenimiento porque libera las dependencias, permitiendo modificar el diseño visual o cambiar la base de datos sin alterar la lógica central.

¿Cuáles son las principales ventajas de los microservicios frente a un diseño monolítico, y qué desafíos introducen?

Las principales ventajas de los microservicios frente a un diseño monolítico son la capacidad de escalar componentes de forma independiente, la posibilidad de realizar despliegues y actualizaciones sin detener la aplicación completa. Por otro lado, los desafíos es que incluye una mayot complejidad operativa en infraestructura y monitoreo, el incremento de la latencia por la comunicación continua a través de la red y una mayor complicación al realizar pruebas.

¿Qué arquitectura consideraría más adecuada para una aplicación como BitTorrent y por qué?

La arquitectura más adecuada para una aplicación como BitTorrent es la arquitectura P2P (Peer-to-Peer), ya que ésta distribuye la carga de los archivos, además de aportar una mayor escalabilidad, ya que el rendimiento aumenta mientras más usuarios descargan y comparten un archivo, por último nos brinda una menor dependencia hacia el servidor central, ya que si un usuario se desconecta, los demás usuarios pueden continuar intercambiando partes del archivo.

¿Cómo influye la escalabilidad horizontal y vertical en la elección de una arquitectura distribuida?

La escalabilidad vertical influye orientado el diseño hacia sistemas monolíticos, ya que consisten en aumentar los recursos de hardware de un único servidor, aunque esto encuentra límites físicos y de costo rápidamente. La escalabilidad horizontal es la fuerza de las arquitecturas distribuidas, pues consiste en agregar más servidores en red para repartir la carga, exigiendo así el diseño de la aplicación de forma modular y gestionando la sincronización de datos entre múltiples nodos.

¿Qué significa transparencia en un sistema distribuido y mencione dos tipos de transparencia?

Transparencia en un sistema distribuido significa que el sistema mantiene la complejidad de la distribución oculta para el usuario, de manera que a pesar de ser diferentes recursos y procesos (ubicados en diferentes computadoras o redes) a simple vista parezca que funciona como un único sistema.

Ejemplos:

1. Transparencia de ubicación: Mantener oculto para el usuario dónde se encuentra físicamente un recurso que está utilizando.
2. Transparencia de acceso: Da libertad al usuario para que pueda utilizar un recurso sin verse afectado de si es local o remoto.

Si un servidor central falla, ¿qué arquitectura continúa funcionando y por qué?

La arquitectura P2P continúa funcionando ya que ésta no depende de un servidor central para ofrecer servicio, ya que como conocemos, cada peer puede actuar como cliente y servidor simultáneamente ya que comparte recursos directamente con otros equipos. En conclusión, P2P es una arquitectura descentralizada.

En su caso de estudio, ¿qué criterios fueron decisivos para elegir la arquitectura propuesta?

8. Declaración de IA

Para el desarrollo de la práctica se utilizó la ayuda de ChatGPT como herramienta para poder estructurar las tablas referentes a las arquitecturas, en el cual se usó el siguiente promt:

"Estamos preparando un taller sobre sistemas distribuidos y necesito comprender mejor las principales arquitecturas: cliente-servidor, peer-to-peer, multicapa (n-tier) y microservicios.

Explícame de forma clara y sencilla qué es cada una, cuáles son sus características principales, cómo funciona la comunicación en cada caso,

sus ventajas y desventajas, y qué aspectos importantes debería conocer sobre estos modelos.”

9. Evaluación

No aplica en esta práctica

10. Bibliografía

Diapositivas